Mathematical Foundations for Data Science & Computational Intelligence

Which Way Do I Turn the Knob?

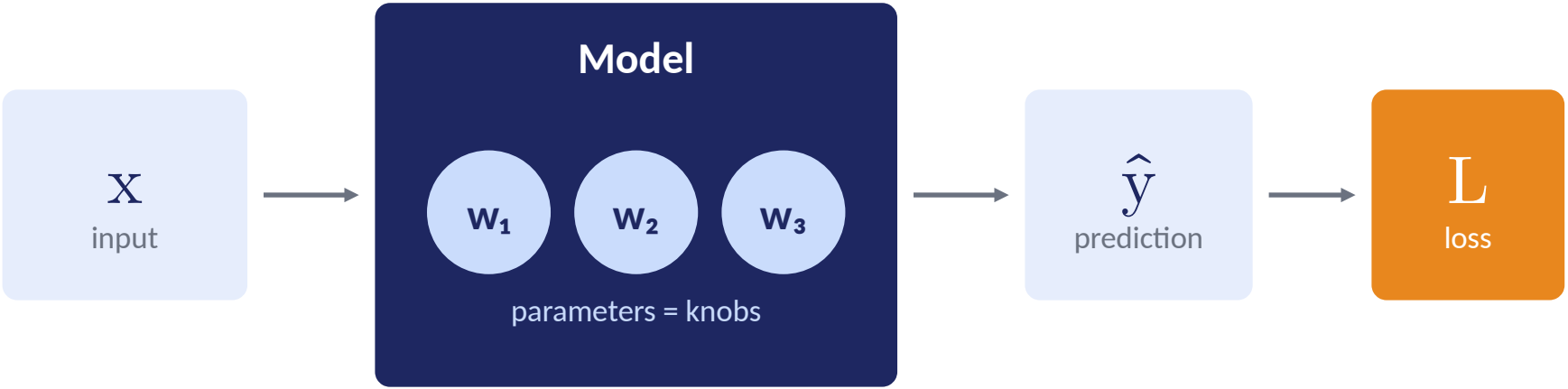
Module 5 · Multivariable Calculus · Session 5.1

Mohsin Raza · Fall 2026

Today

1	Opening: a model is a function with knobs	5 min
2	Hook: nudge and observe	15 min
3	Single-variable recall: the derivative as sensitivity	25 min
4	Second derivative & curvature	10 min
5	Partial derivatives: one knob at a time	15 min
6	Where you'll meet this at work	5 min
7	Wrap-up	5 min

A Model Is a Function with Knobs



Training = turning the knobs until the loss goes down.
For the next two weeks: which way do you turn them, and how much does the loss care?

The Smallest Possible Model

$$\hat{y} = w \cdot x$$

One data point: x = 2, y = 3

Current knob: w = 1

Loss: $L(w) = (w \cdot x - y)^2$

Should w go up or down?

Forget calculus. You're a programmer and this function is in your code. How would you find out?

Nudge and Observe

$L(1)$	$L(1.01)$	slope $\approx$
1.0000	**0.9604**	**−3.96**

```
slope = (L(w + h) - L(w)) / h      # h = 0.01
```

Negative slope → increase w.
That fraction is the derivative, computed the way a programmer would. Exact value: −4.

QUESTION FOR THE ROOM

Now the model has 3 knobs. How many times do you run it?

The Cost of Nudging

Model	Parameters	Runs per gradient	At 1 ms per run
Toy model	1	2	instant
ResNet-50 (images)	~25.6 M	~25.6 M	~7 hours
GPT-2 small	~124 M	~124 M	**~34 hours**

~34 h

for ONE gradient step of GPT-2 small, by nudging

Training takes hundreds of thousands of steps. Yet these models train.
So nobody computes gradients by nudging. What they do instead: Session 5.4.

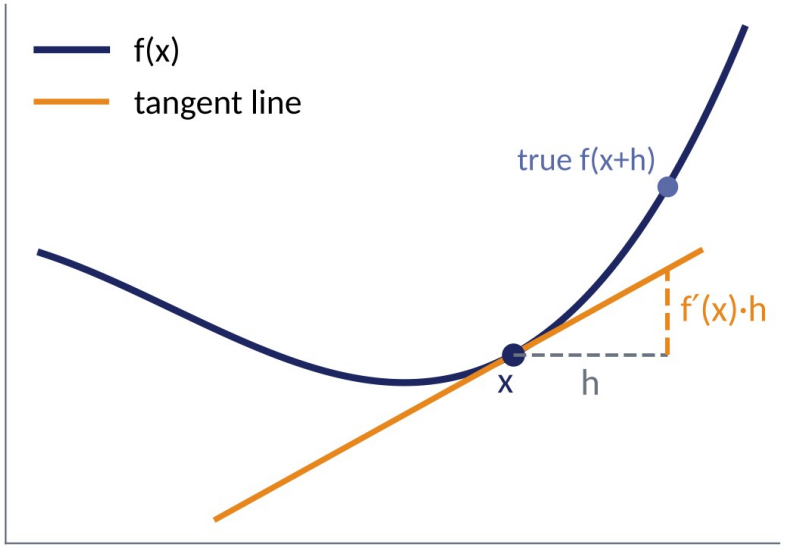
The Derivative Is a Sensitivity

$$f(x + h) \approx f(x) + f'(x) \cdot h$$

Sensitivity: if the input moves by h, the output moves by about $f'(x) \cdot h$.

Local linear approximation: zoom in far enough and a smooth function looks like a straight line.

Not a limit to memorize. A tool for predicting change.



The Only Rules We Need

Power

$$(x^n)' = n\,x^{n-1}$$

Sum & constant multiple

$$(a\,f + g)' = a\,f' + g'$$

Product

$$(f\,g)' = f'g + f\,g'$$

Chain

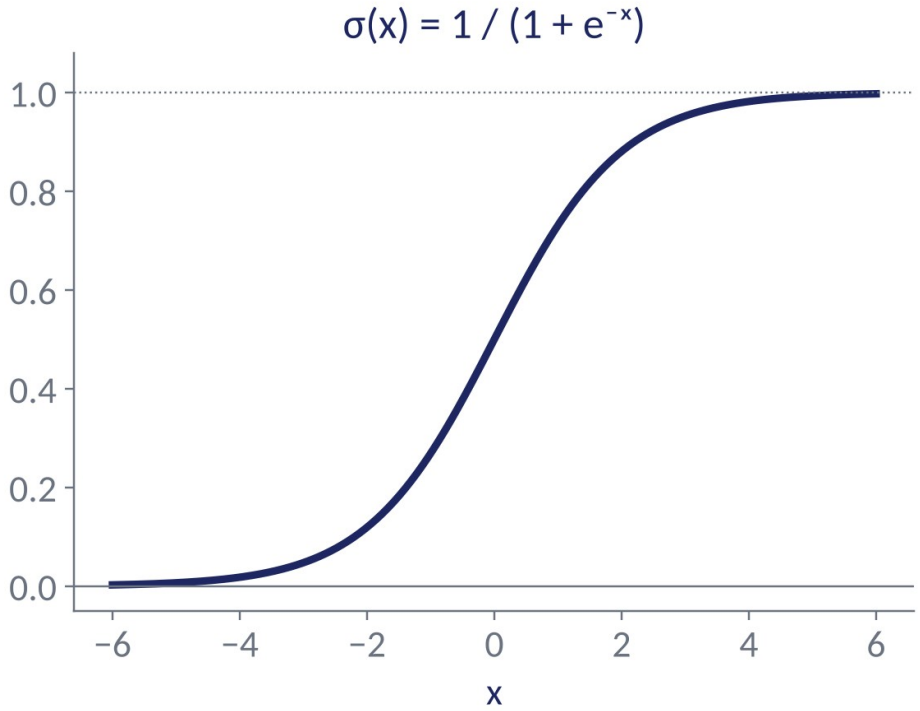
$$(f(g(x)))' = f'(g(x)) \cdot g'(x)$$

This one rule is backpropagation.

The ML Function Zoo

Function	Derivative	Where it shows up
e^x	e^x	softmax, probabilities (Module 6)
$\ln x$	$1 / x$	log-likelihood, cross-entropy (Modules 6, 9)
$\sigma(x) = 1 / (1 + e^{-x})$	$\sigma(x)(1 - \sigma(x))$	classic activation, logistic regression
$\tanh x$	$1 - \tanh^2 x$	activation
$\mathrm{ReLU}(x) = \max(0, x)$	1 if $x > 0$, 0 if $x < 0$	the default modern activation

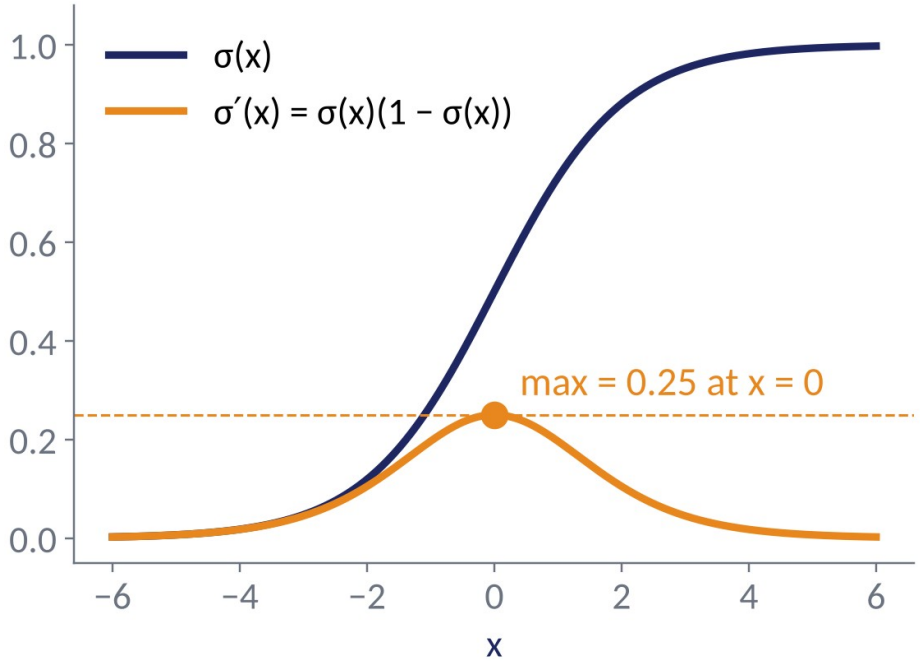
How Steep Can the Sigmoid Get?



What's the largest value σ′(x) can take?

Hint: where is σ steepest?
Use σ′ = σ(1 − σ).

Remember This Number



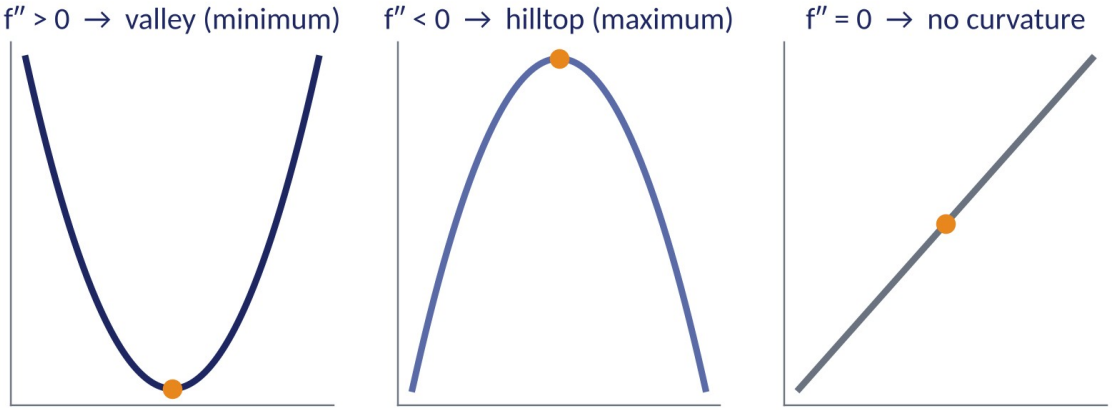
0.25

$$\sigma(0) = \tfrac{1}{2}$$
$$\sigma'(0) = \tfrac{1}{2} \cdot \tfrac{1}{2} = 0.25$$

The largest the sigmoid's derivative can ever be. In Session 5.4 it explains why deep networks were stuck for a decade.

Second Derivative: Curvature

$$f(x + h) \approx f(x) + f'(x)\,h + \tfrac{1}{2} f''(x)\,h^2$$



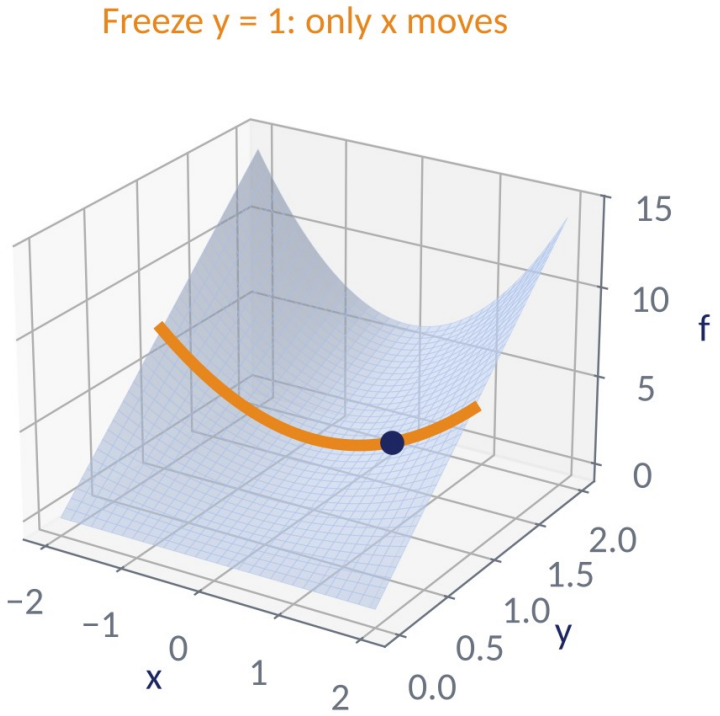
f′ says which way is downhill. f″ says how curved the valley is. **Session 5.3: the same formula with vectors, and f″ becomes a matrix (the Hessian).**

Partial Derivatives: Freeze the Others

$$f(x, y) = x^2y + 3y$$

Find ∂f/∂x and ∂f/∂y.

A partial derivative is the nudge from earlier, applied to one knob while every other knob stays frozen.



Frozen Means Constant

$$\partial f/\partial x = 2xy$$

y is frozen, so 3y is a constant. Its derivative is 0, not 3.

$$\partial f/\partial y = x^2 + 3$$

x is frozen, so x^2 is just a coefficient, like the 3 in 3y.

“Frozen” means it behaves exactly like the number 5 would.

Same Idea, in Model Form

$$L(w,\ b) = (w\ x + b - y)^2$$ one data point, two knobs

$$\partial L/\partial w = 2(w\ x + b - y) \cdot x$$
$$\partial L/\partial b = 2(w\ x + b - y) \cdot 1$$

error signal = 2(prediction − truth), shared by both knobs
× how much each knob touched the prediction (x for w, 1 for b)

That pattern is backpropagation in miniature.

Where You'll Meet This at Work

1 Sensitivity analysis

Pricing, risk, demand forecasts: which input is the output most sensitive to? That's a partial derivative, and it's the question every stakeholder actually asks.

2 Gradient checking

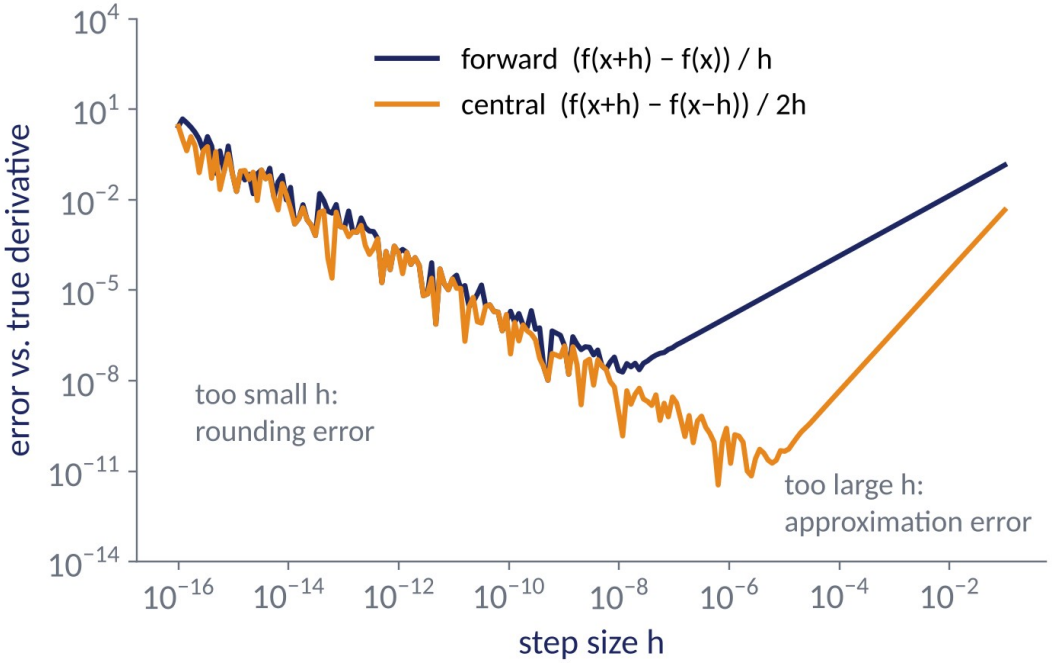
Hand-written gradients are unit-tested against today's nudge-and-observe method. The "naive" method is a real debugging tool, just never the training method.

Next: All the Knobs at Once

Thursday · Session 5.2: Gradient, Jacobian, Chain Rule

Questions?

Why Not Just Make h Tiny?



Real data: f(x) = e^x at x = 1, double precision.

Too small: f(x + h) and f(x) agree in almost every digit, and subtracting them wipes the digits out.

Central difference is far more accurate at its best h (≈10^{-12} vs ≈10^{-8}).

Same family as Day 1's overflow: the machine's number set runs out of precision.

Deriving $\sigma' = \sigma(1 - \sigma)$

$$\sigma(x) = \left(1 + e^{-x}\right)^{-1}$$

$$\sigma'(x) = -\left(1 + e^{-x}\right)^{-2} \cdot \left(-e^{-x}\right) = \frac{e^{-x}}{\left(1 + e^{-x}\right)^2} \qquad \text{(chain rule)}$$

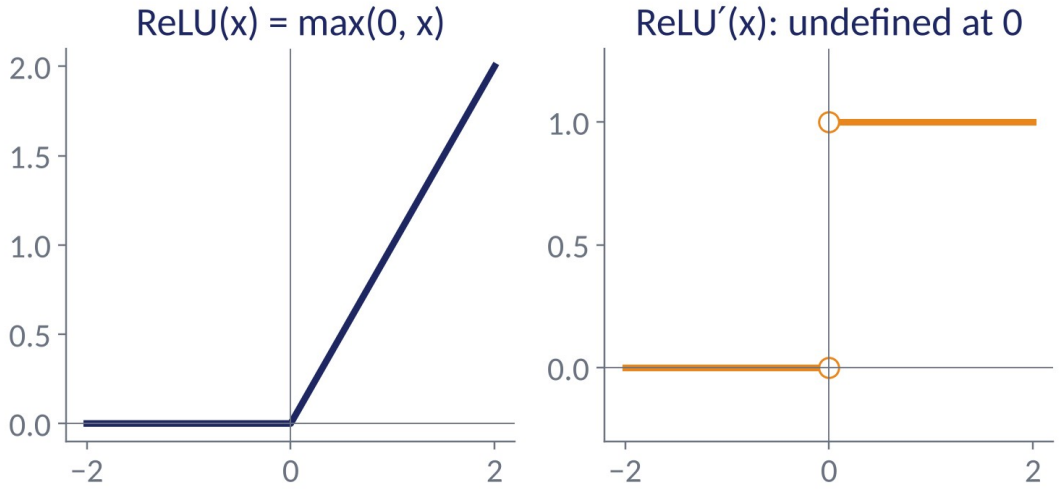
$$= \underbrace{\frac{1}{1 + e^{-x}}}_{\sigma(x)} \cdot \underbrace{\frac{e^{-x}}{1 + e^{-x}}}_{1 - \sigma(x)} = \sigma(x)\left(1 - \sigma(x)\right)$$

Three lines, and the only rule used is the chain rule. The last step uses $e^{-x} / (1 + e^{-x}) = 1 - \sigma(x)$.

When the Derivative Doesn't Exist

Formal definition (optional rigor):

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$



ReLU has a kink at 0: the limit from the left (0) and from the right (1) disagree, so there's no derivative there. Frameworks just pick a value (PyTorch uses 0). Hitting exactly 0 in floating point is rare, so it doesn't matter in practice.